

Tutorial de ciencia de datos en VS Code

Este tutorial muestra el uso de Visual Studio Code y la extensión de Microsoft Python con bibliotecas comunes de ciencia de datos para explorar un escenario básico. Específicamente, utilizando datos de pasajeros del Titanic, aprenderá a configurar un entorno de ciencia de datos, importar y limpiar datos, crear un modelo de aprendizaje automático para predecir la supervivencia a bordo del Titanic y evaluar la precisión del modelo generado.

Prerrequisitos

Se requieren las siguientes instalaciones para completar este tutorial. Asegúrate de instalarlas si aún no lo has hecho.

- [Código de Visual Studio](#)
- La [extensión de Python para VS Code](#) y la [extensión de Jupyter para VS Code](#) están disponibles en Visual Studio Marketplace. Para obtener más información sobre la instalación de extensiones, consulte [Extension Marketplace](#) . Ambas extensiones son publicadas por Microsoft.
- [Miniconda con la última versión de Python](#)

Nota : Si ya tiene instalada la distribución completa de Anaconda, no necesita instalar Miniconda. Si prefiere no usar Anaconda ni Miniconda, puede crear un entorno virtual de Python e instalar los paquetes necesarios para el tutorial con pip. Si sigue esta ruta, deberá instalar los siguientes paquetes: pandas, jupyter, seaborn, scikit-learn, keras y tensorflow.

Configurar un entorno de ciencia de datos

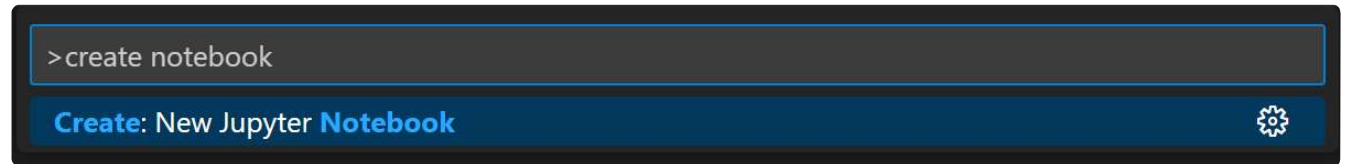
Visual Studio Code y la extensión de Python ofrecen un excelente editor para escenarios de ciencia de datos. Gracias a la compatibilidad nativa con Jupyter Notebooks y Anaconda, es fácil comenzar. En esta sección, creará un espacio de trabajo para el tutorial, un entorno de Anaconda con los módulos de ciencia de datos necesarios y un Jupyter Notebook que usará para crear un modelo de aprendizaje automático.

Comience creando un entorno de Anaconda para el tutorial de ciencia de datos. Abra el símbolo del sistema de Anaconda y ejecútelo `conda create -n myenv python=3.10 pandas jupyter seaborn scikit-learn keras tensorflow` para crear un entorno llamado **myenv** . Para obtener más información sobre la creación y administración de entornos de Anaconda, consulte la [documentación de Anaconda](#) .

A continuación, crea una carpeta en una ubicación conveniente para que sirva como tu espacio de trabajo de VS Code para el tutorial; nómbrala `hello_ds`.

Abra la carpeta del proyecto en VS Code ejecutando VS Code y usando el comando **Archivo > Abrir carpeta**. Puede abrir la carpeta con total seguridad, ya que la creó usted mismo.

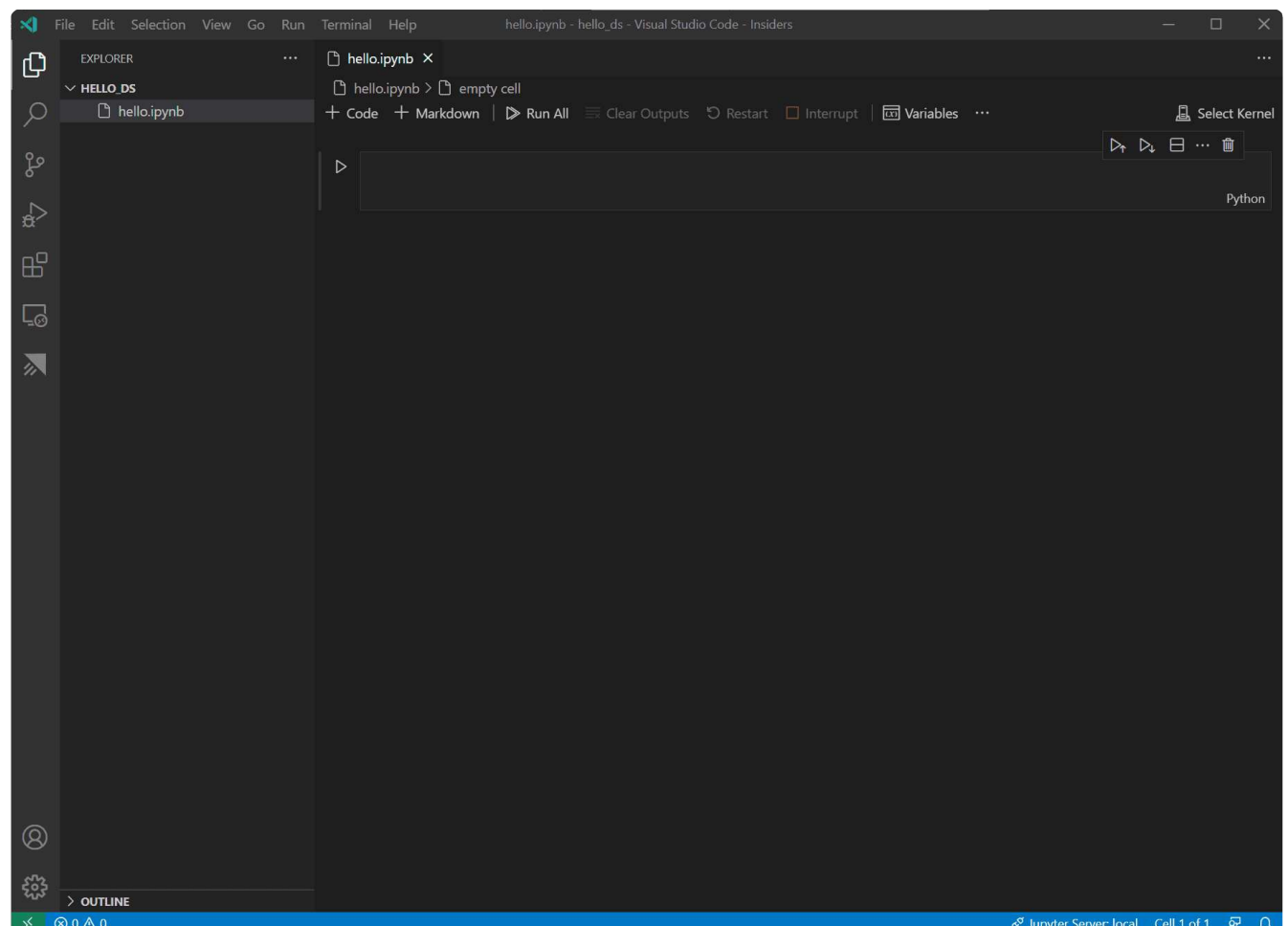
Una vez iniciado VS Code, cree el Jupyter Notebook que se usará para el tutorial. Abra la paleta de comandos (`Ctrl+Mayús+P`) y seleccione **Crear: Nuevo Jupyter Notebook**.



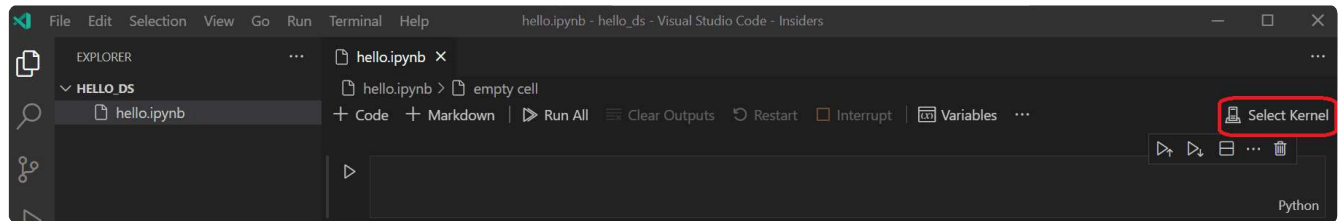
Nota : Como alternativa, desde el Explorador de archivos de VS Code, puede utilizar el ícono Nuevo archivo para crear un archivo Notebook llamado `hello.ipynb`.

Guarde el archivo como `hello.ipynb` usando **Archivo > Guardar como**

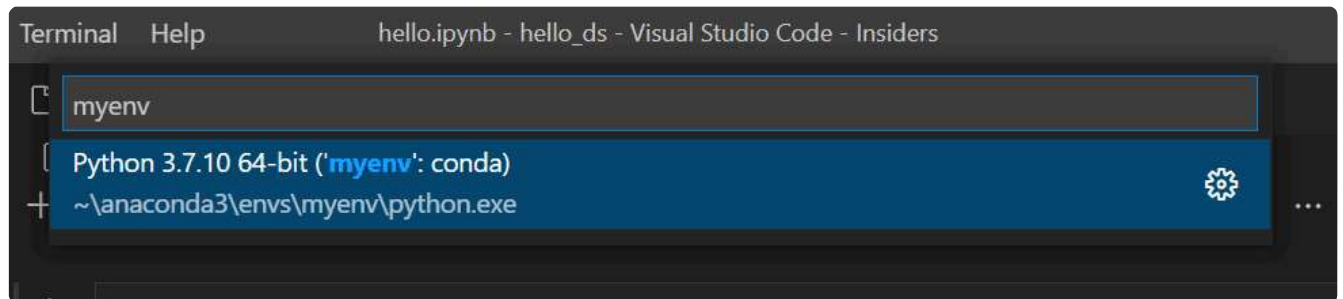
Una vez creado el archivo, debería ver el [Jupyter Notebook](#) abierto en el editor de notebooks. Para obtener más información sobre la compatibilidad nativa con Jupyter Notebooks, consulte el tema sobre [Jupyter Notebooks](#).



Ahora seleccione **Seleccionar Kernel** en la parte superior derecha del cuaderno.



Elija el entorno Python que creó anteriormente en el que ejecutará su kernel.



Para administrar su entorno desde la terminal integrada de VS Code, ábrala con (`Ctrl+``). Si su entorno no está activado, puede hacerlo como lo haría en su terminal (`conda activate myenv`).

Preparar los datos

Este tutorial utiliza el [conjunto de datos del Titanic](https://hbiostat.org/data) , disponible en [OpenML.org](https://openml.org) , obtenido del Departamento de Bioestadística de la Universidad de Vanderbilt en <https://hbiostat.org/data> . Los datos del Titanic proporcionan información sobre la supervivencia de los pasajeros y sus características, como la edad y la clase del billete. Con estos datos, el tutorial establecerá un modelo para predecir si un pasajero habría sobrevivido al hundimiento del Titanic. Esta sección muestra cómo cargar y manipular datos en su cuaderno de Jupyter.

Para comenzar, descargue los datos del Titanic desde hbiostat.org como un archivo CSV (enlaces de descarga en la parte superior derecha) `titanic3.csv` y guárdelo en la `hello_ds` carpeta que creó en la sección anterior.

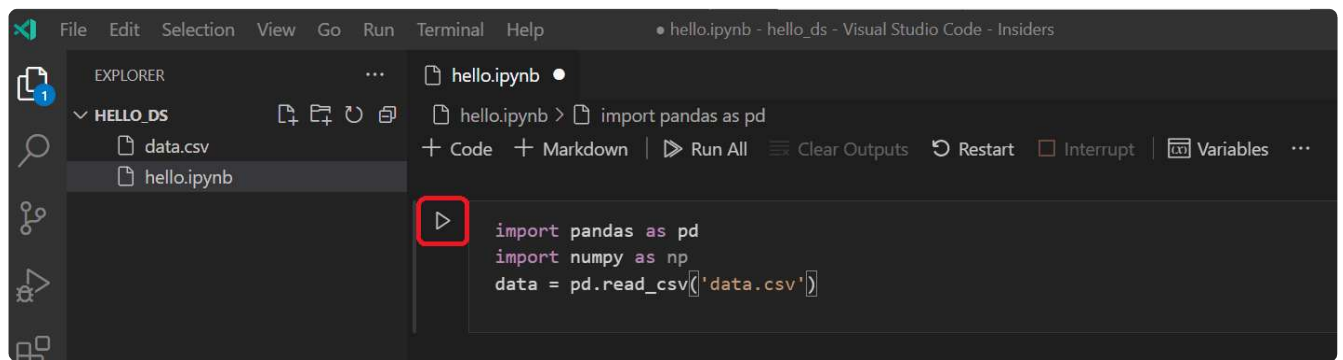
Si aún no ha abierto el archivo en VS Code, abra la `hello_ds` carpeta y el cuaderno Jupyter (`hello.ipynb`), yendo a **Archivo > Abrir carpeta** .

En tu cuaderno de Jupyter, empieza importando las bibliotecas `pandas` y `numpy` , dos bibliotecas comunes para manipular datos, y cargando los datos de Titanic en un `DataFrame` de `pandas` . Para ello, copia el código a continuación en la primera celda del cuaderno. Para obtener más información sobre cómo trabajar con cuadernos de Jupyter en VS Code, consulta la documentación sobre [cómo trabajar con cuadernos de Jupyter](#) .

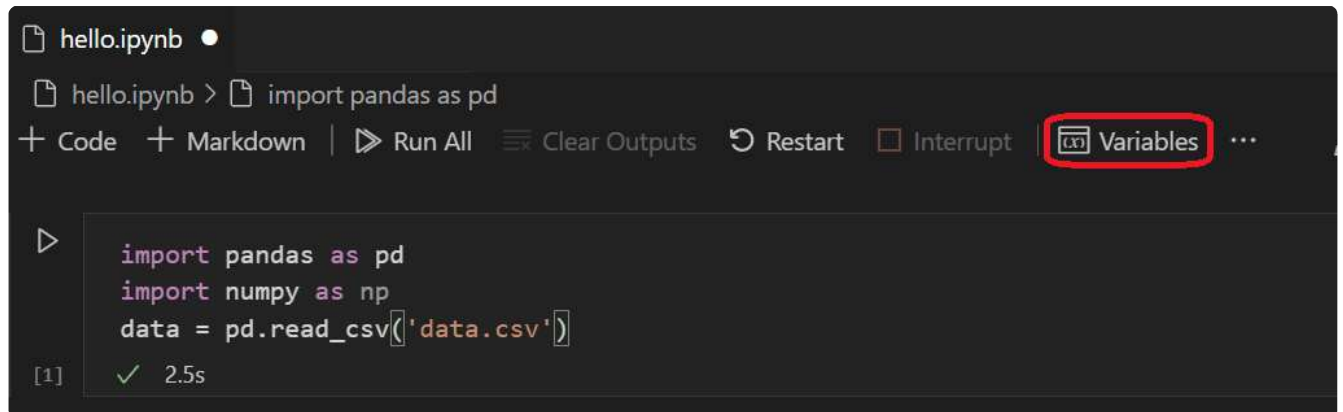
```
import pandas as pd
import numpy as np
data = pd.read_csv('titanic3.csv')
```

Copiar

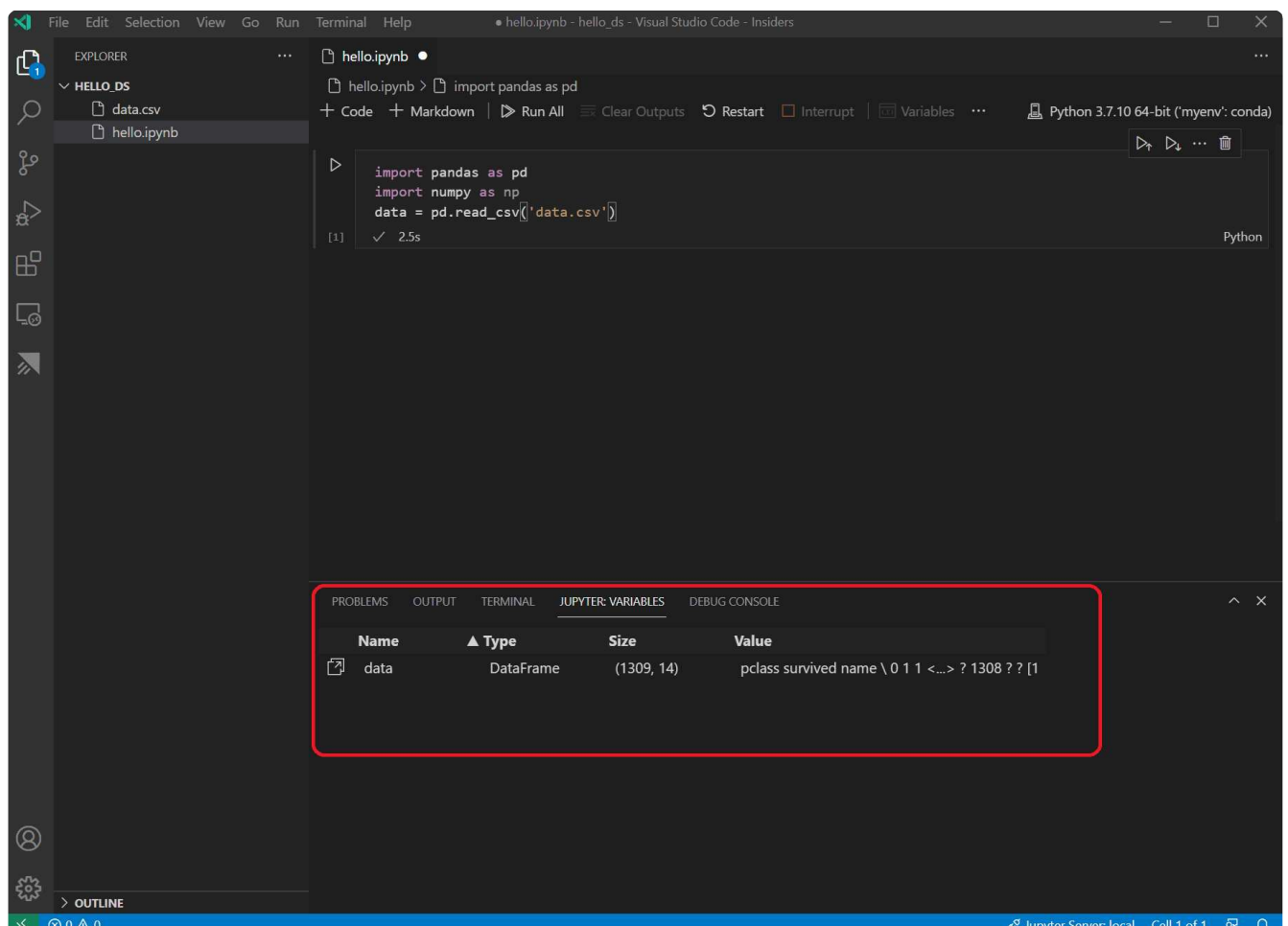
Ahora, ejecute la celda utilizando el ícono Ejecutar celda o el atajo `Shift+Enter` .



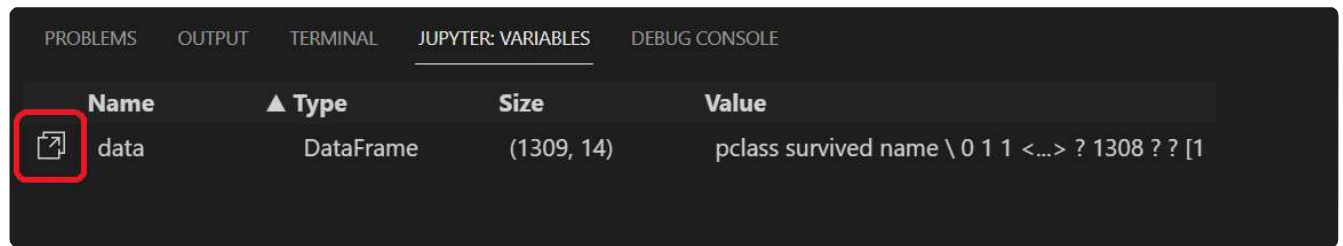
Una vez finalizada la ejecución de la celda, puede ver los datos cargados mediante el Explorador de Variables y el Visor de Datos. Primero, seleccione el icono de **Variables** en la barra de herramientas superior del cuaderno.



Se abrirá el panel **JUPYTER: VARIABLES** en la parte inferior de VS Code. Contiene una lista de las variables definidas hasta el momento en el kernel en ejecución.



Para ver los datos en el Pandas DataFrame previamente cargado, seleccione el ícono Visor de datos a la izquierda de la `data` variable.



Utilice el Visor de Datos para ver, ordenar y filtrar las filas de datos. Tras revisar los datos, puede ser útil graficar algunos aspectos para visualizar las relaciones entre las diferentes variables.

A screenshot of the 'Data Viewer' interface. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL', 'JUPYTER: VARIABLES' (selected), and 'DEBUG CONSOLE'. Below the tabs is a table with 11 columns: 'index', 'pclass', 'surviv...', 'name', 'sex', 'age', 'sibsp', 'parch', 'ticket', and 'fare'. The table contains 15 rows of data. The first row shows index 0, pclass 1, survived 1, name 'Allen, M...', sex 'female', age 29, sibsp 0, parch 0, ticket 24160, and fare 211.3375. The second row shows index 1, pclass 1, survived 1, name 'Allison, ...', sex 'male', age 0.9167, sibsp 1, parch 2, ticket 113781, and fare 151.55. The third row shows index 2, pclass 1, survived 0, name 'Allison, ...', sex 'female', age 2, sibsp 1, parch 2, ticket 113781, and fare 151.55. The fourth row shows index 3, pclass 1, survived 0, name 'Allison, ...', sex 'male', age 30, sibsp 1, parch 2, ticket 113781, and fare 151.55. The fifth row shows index 4, pclass 1, survived 0, name 'Allison, ...', sex 'female', age 25, sibsp 1, parch 2, ticket 113781, and fare 151.55. The sixth row shows index 5, pclass 1, survived 1, name 'Anderson...', sex 'male', age 48, sibsp 0, parch 0, ticket 19952, and fare 26.55. The seventh row shows index 6, pclass 1, survived 1, name 'Andrews, ...', sex 'female', age 63, sibsp 1, parch 0, ticket 13502, and fare 77.9583. The eighth row shows index 7, pclass 1, survived 0, name 'Andrews, ...', sex 'male', age 39, sibsp 0, parch 0, ticket 112050, and fare 0. The ninth row shows index 8, pclass 1, survived 1, name 'Appleton...', sex 'female', age 53, sibsp 2, parch 0, ticket 11769, and fare 51.4792. The tenth row shows index 9, pclass 1, survived 0, name 'Artagave...', sex 'male', age 71, sibsp 0, parch 0, ticket 'PC 17609', and fare 49.5042. The eleventh row shows index 10, pclass 1, survived 0, name 'Astor, C...', sex 'male', age 47, sibsp 1, parch 0, ticket 'PC 17757', and fare 227.525. The twelfth row shows index 11, pclass 1, survived 1, name 'Astor, M...', sex 'female', age 18, sibsp 1, parch 0, ticket 'PC 17757', and fare 227.525. The thirteenth row shows index 12, pclass 1, survived 1, name 'Aubart, ...', sex 'female', age 24, sibsp 0, parch 0, ticket 'PC 17477', and fare 69.3. The fourteenth row shows index 13, pclass 1, survived 1, name 'Barber, ...', sex 'female', age 26, sibsp 0, parch 0, ticket 19877, and fare 78.85. The fifteenth row shows index 14, pclass 1, survived 1, name 'Barkwort...', sex 'male', age 80, sibsp 0, parch 0, ticket 27042, and fare 30. Below the table, there is a summary bar with the same information as the Jupyter Variables panel: 'Name: data', 'Type: DataFrame', 'Size: (1309, 14)', and 'Value: pclass survived name \ 0 1 1 <...> ? 1308 ? ? [1'.

Como alternativa, puede usar la experiencia de visualización de datos que ofrecen otras extensiones como [Data Wrangler](#) . Esta extensión ofrece una interfaz de usuario completa que muestra información sobre sus datos y le ayuda a realizar perfiles de datos, controles de calidad, transformaciones y mucho más. Obtenga más información sobre [Data Wrangler en nuestra documentación](#) .

Antes de graficar los datos, debe asegurarse de que no presenten ningún problema. Si observa el archivo csv del Titanic, notará que se usó un signo de interrogación (" ? ") para identificar las celdas donde no había datos disponibles.

Si bien Pandas puede leer este valor en un DataFrame, el resultado para una columna como `age` es que su tipo de dato se establecerá en `objeto` en lugar de un tipo de dato numérico, lo que resulta problemático para los gráficos.

Este problema se puede corregir reemplazando el signo de interrogación con un valor faltante que Pandas pueda interpretar. Agrega el siguiente código a la siguiente celda de tu notebook para reemplazar los signos de interrogación en las columnas de `edad` y `tarifa` con el valor `NaN` de `numpy` . Ten en cuenta que también debemos actualizar el tipo de dato de la columna después de reemplazar los valores.

Consejo : Para agregar una nueva celda, puede usar el icono de insertar celda, ubicado en la esquina inferior izquierda de una celda existente. También puede usar la tecla `Esc` para ingresar al modo comando, seguida de la tecla `B`.

```
data.replace('?', np.nan, inplace= True)
data = data.astype({"age": np.float64, "fare": np.float64})
```

Copiar

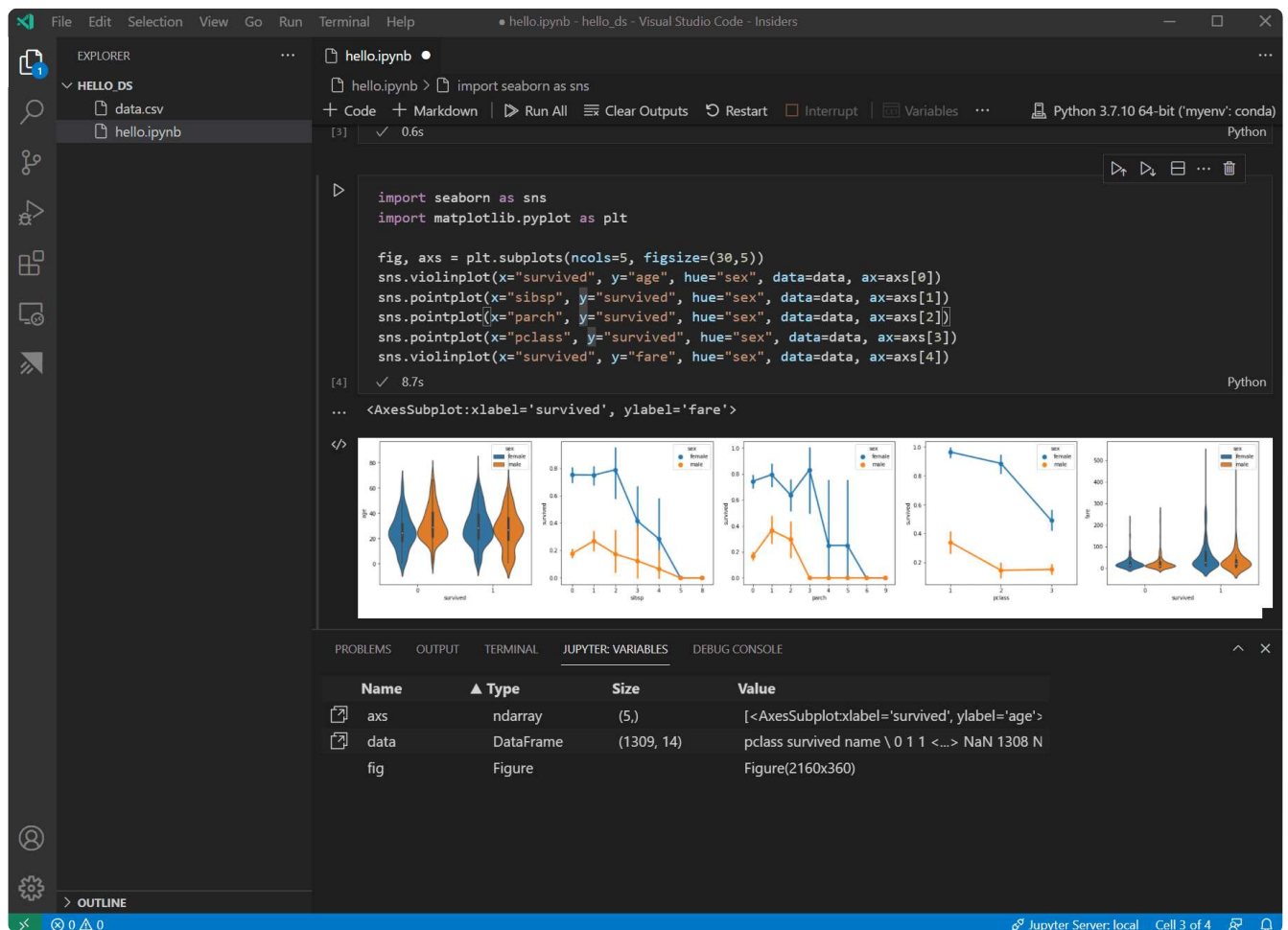
Nota : Si alguna vez necesita ver el tipo de datos que se ha utilizado para una columna, puede utilizar el atributo `dtypes` de `DataFrame` .

Ahora que los datos están bien organizados, puede usar [seaborn](#) y [matplotlib](#) para ver cómo se relacionan ciertas columnas del conjunto de datos con la supervivencia. Agregue el siguiente código a la siguiente celda de su cuaderno y ejecútelo para ver los gráficos generados.

```
import seaborn as sns
import matplotlib.pyplot as plt

fig, axs = plt.subplots(ncols=5, figsize=(30,5))
sns.violinplot(x="survived", y="age", hue="sex", data=data, ax=axs[0])
sns.pointplot(x="sibsp", y="survived", hue="sex", data=data, ax=axs[1])
sns.pointplot(x="parch", y="survived", hue="sex", data=data, ax=axs[2])
sns.pointplot(x="pclass", y="survived", hue="sex", data=data, ax=axs[3])
sns.violinplot(x="survived", y="fare", hue="sex", data=data, ax=axs[4])
```

Copiar



Consejo : Para copiar rápidamente su gráfico, coloque el cursor sobre la esquina superior derecha y haga clic en el botón **"Copiar al portapapeles"** . También puede ver mejor los detalles del gráfico haciendo clic en el botón **"Expandir imagen"** .



Estos gráficos son útiles para visualizar algunas de las relaciones entre la supervivencia y las variables de entrada de los datos, pero también es posible usar **pandas** para calcular correlaciones. Para ello, todas las variables utilizadas deben ser numéricas para el cálculo de la correlación, y actualmente el género se almacena como una cadena. Para convertir esos valores de cadena a enteros, agregue y ejecute el siguiente código.

```
data.replace({'male': 1, 'female': 0}, inplace=True)
```

[Copiar](#)

Ahora, puede analizar la correlación entre todas las variables de entrada para identificar las características que serían las mejores entradas para un modelo de aprendizaje automático. Cuanto más cercano esté un valor a 1, mayor será la correlación entre el valor y el resultado. Utilice el siguiente código para correlacionar la relación entre todas las variables y la supervivencia.

```
data.corr(numeric_only=True).abs()[["survived"]]
```

[Copiar](#)

```
data.corr().abs()[["survived"]]
```

[6] ✓ 0.3s

	survived
pclass	0.312469
survived	1.000000
sex	0.528693
age	0.055513
sibsp	0.027825
parch	0.082660
fare	0.244265

Al observar los resultados de la correlación, notará que algunas variables, como el género, tienen una correlación bastante alta con la supervivencia, mientras que otras, como los parientes (sibsp = hermanos o cónyuge, parch = padres o hijos), parecen tener poca correlación.

Supongamos que **sibsp** y **parch** están relacionados en cuanto a su efecto sobre la supervivencia, y agrupémoslos en una nueva columna llamada "parientes" para ver si su combinación tiene una mayor correlación con la supervivencia. Para ello, comprobaremos si, para un pasajero dado, el número de **sibsp** y **parch** es mayor que 0 y, de ser así, podemos afirmar que tenía un familiar a bordo.

Utilice el siguiente código para crear una nueva variable y columna en el conjunto de datos llamado `relatives` y verificar la correlación nuevamente.

```
data['relatives'] = data.apply (lambda row: int((row['sibsp'] + row['parch']) > 0),  
1)  
data.corr(numeric_only=True).abs()[["survived"]]
```

[Copiar](#)

```
data['relatives'] = data.apply (lambda row: int((row['sibsp'] + row['parch']  
data.corr().abs()[["survived"]])  
[7] ✓ 0.2s  
...  
survived  
pclass 0.312469  
survived 1.000000  
sex 0.528693  
age 0.055513  
sibsp 0.027825  
parch 0.082660  
fare 0.244265  
relatives 0.201719
```

Observará que, de hecho, al analizar si una persona tenía parientes, en comparación con cuántos, existe una mayor correlación con la supervivencia. Con esta información, puede eliminar del conjunto de datos las columnas **sibsp** y **parch** de bajo valor , así como las filas con valores **NaN** , para obtener un conjunto de datos que pueda usarse para entrenar un modelo.

```
data = data[['sex', 'pclass', 'age', 'relatives', 'fare', 'survived']].dropna()
```

Copiar

Nota : Aunque la edad tenía una correlación directa baja, se mantuvo porque parece razonable que aún pudiera tener correlación junto con otras entradas.